CS3230 Design and Analysis of Algorithms	July 11, 2020

Solution *Sketches* to Problem Set 3

Name: Divesh Aggarwal

Instruction

This tutorial contains two kinds of questions - questions marked with an asterisk (*) are part of the assignment, and you need to submit your solutions to these questions; solutions to unmarked questions need not be submitted, but you should still attempt them. Your assignment solution will be graded. Submit your assignment solution by uploading to the LumiNUS. Solutions typeset using LaTeXare encouraged but not necessary. You may also submit scanned handwritten solutions, but they must be clearly readable. Solutions that are not readable may receive only partial marks. Late submissions will receive no marks. This assignment solution will be due by 11:59 pm, July 11, 2020 (Saturday).

Problem 3-1 (*Minimize the number of coins) 10 Points

Assume you live in a country, which has k different currency coins having values $b_1 = 1 < b_2 < \ldots < b_k$ cents. You just bought an amazing algorithms textbook costing $T > 0$ cents. Assume that you have an unlimited number of coins of each of the k values, and you want to provide *exact* change equaling T cents using the minimal number of coins c. For example, if $k = 2, b_1 = 1, b_2 = 3$ and $T = 5$, the optimal solution is $3 + 1 + 1 = 5$, using only $c = 3$ coins, since the other solution $1 + 1 + 1 + 1 + 1$ uses 5 coins.

Consider the following greedy algorithm for solving this problem: "Find the largest valued coin whose value v is less than or equal to T. Add this coin to your solution and recurse on $(T - v)$."

(a) Give an example with $k = 3$ (i.e., values of T, b_1, b_2, b_3) for which the above mentioned algorithm does not give the optimal solution. (3 Points)

Solution: Let $T = 8$, and $b_1 = 1$, $b_2 = 4$, and $b_3 = 6$. Then the greedy algorithm above gives $c = 3$ coins $(8 = 6 + 1 + 1)$, while the smallest solution is 2 coins $(8 = 4 + 4)$. □

(b) Now assume that $b_i = 3^{i-1}$ for $1 \leq i \leq k$. Use Local Swap to argue that in this case, the greedy algorithm given solve the correct answer. (7 Points)

(**Hint**: First show that $2 \cdot (b_1 + \ldots + b_{i-1}) < b_i$. How can you use this inequality?)

Solution: Consider any optimal solution that differs from the solution given by the greedy algorithm, and let the coins chosen be $v_1 \geq v_2 \geq \cdots v_n$. Let the greedy algorithm choose coins $u_1 \geq u_2 \geq \cdots \geq u_m$. Note that each $u_i, v_i \in \{3^0, \ldots, 3^{k-1}\}$. Consider the smallest index i such that $v_i < u_i$. There exists such an index because of the assumption that the optimal

solution differs from the greedy solution, which always chooses the largest possible coin at each step. Thus $v_i = 3^j$ and $u_i = 3^\ell$ for $j \leq \ell - 1$. This implies that $v_i + \cdots + v_n \geq 3^\ell$. We know that $v_i, \ldots, v_n \leq 3^j$. Thus, we claim that there are at least three coins in $\{v_i, \ldots, v_n\}$ of value 3^t for some $t \leq j$. This is because

$$2(3^0 + \ldots + 3^j) = 3^{j+1} - 1 < 3^\ell \ .$$

These three coins of value 3^t can be replaced by a single coin of value 3^{t+1} thus contradicting the optimality of the solution.

The running time of this algorithm is $O(T/3^{k-1} + k)$. □

Problem 3-2 (Uninterrupted Health Insurance)

You live in a country with very high medical costs. You also anticipate undergoing a super-expensive medical treatment for the time period $[S, F)$. The interval $[S, F)$ represents numbers greater or equals to S and strictly less than F. Luckily, as long as you have at least one job, you will have a cheap health insurance which will fully cover your expensive treatment. You are also very smart, so you get n job offers, where the i-th job is for the period $[s_i, f_i)$, so that the union of $[s_i, f_i)$ fully covers the desired interval $[S, F)$. For simplicity, assume the jobs are already sorted by their starting times, so that $s_1 \leq s_2 \leq \ldots \leq s_n$.

Of course, one possible solution is to take all the jobs (it's OK to have multiple jobs at the same time). However, being a naturally lazy person, you would like to take the smallest number of jobs which would cover the desired interval $[S, T)$.

(a) Design an efficient $O(n)$ greedy algorithm for this problem. For partial credit, give $O(n^2)$ algorithm. Do not forget to carefully argue the correctness of your algorithm.

Solution: Here is the insight which will give us our greedy algorithm. Let A be the set of all jobs that start before S and let $a \in A$ be the job in A that ends the latest. Then any solution to the Health Insurance problem must have at least one job in A. (Otherwise your not insured at the beginning of your treatment.) And if the solution has a job in A then we can always replace that job with a and we still have a valid solution which as good as the original. What's more we can completely remove any other jobs in A from the solution since they are completely covered by job a anyway. This operation can only improve our solution. So basically what i just argued is that given any optimal solution Z to the Health Insurance problem. We can always find an equivalent solution to Z that has only job a from the set A.

So that is going to be our greedy choice. In other words, on the highest level the algorithm does the following.

1. Find $[s_a, f_a) = a \in A$.
2. Add a to the solution, set $S = f_a$ and repeat until $S \geq F$.

To prove that this algorithm is correct we use induction on how many jobs are in the optimal solution. For the base case we argue that if the optimal solution has only one job then the algorithm works. Well, from the discussion above we know that one such optimal solution is

the set $\{a\}$ which is exactly what the greedy algorithm outputs. So that case is done. Now we assume that the algorithm can find optimal solutions up to size $i-1$.

Now we consider problems who's optimal solution has i jobs. Let $a = [s_a, f_a)$ and let T be an optimal solution that contains a. (We know such a T exists by the above argument.) Then the crucial step is to argue that by removing a from T we get an optimal solution to the subproblem $[f_a, F)$. If we can show this we are done because by assumption our algorithm solves that subproblem optimally and also adds a to that solution so it produces something that is just as good as T for problem $[S, F)$. is this some recursion thing

Suppose for a moment that $T' = T \backslash \{a\}$ is not optimal for $[f_a, F)$ and let T'' be the a better (optimal) solution for that segment. Then we could solve the entire problem $[S, F)$ with $T'' \cup \{a\}$ which is strictly a better solution then T. That contradicts the optimality of T though so it must be that T' is also optimal for $[f_a, F)$. This concludes the proof that our greedy algorithm produces optimal solutions.

Here is the pseudocode for the greedy algorithm. It takes in parameters S, F and array J of length n. The entries of J are pairs of integers where $J[i].s$ is the start time for i-th job and $J[i].f$ is it's end time. Note that J is sorted by start times. We also assume that there is always at least one valid solution for the given inputs.

```
HealthInsurrance(S, F, J, n)
    i = 1; T = ∅
    While S < F
        max = i
        While (J[i].s ≤ S) ∧ (i ≤ n)
            If J[i].f > J[max].f Then
                max = i
            i = i + 1
        T = T ∪ i
        S = J[i].f_i
    Return T
```

To compute the runtime of this algorithm it suffices to see that the if-statement is run at most once for every value of $i \in \{1 \ldots n\}$ since i ever changed by being incremented by 1. So the runtime of the total algorithm is $O(n)$. $\square$

(b) Assume you can hold at most two jobs at the same time (e.g., each job only has a day shift and a night shift, and you have to be at the job). Does your solution from part (a) still work in this case? If yes, prove it. Else, give a counter-example.

Solution: The solution from part (a) still works. This follows from the claim that no optimal solution to the problem in part (a) will ever have more then 2 jobs overlapping each other at a time.

We prove this claim by contradiction. Suppose we have an optimal solution T that contains three jobs x, y and z which over lap each other at some point such that x starts first, then

y and finally z. Also let job $q \in \{x, y, z\}$ be the job that ends the latest. Then I claim that we can simply remove any the jobs in $\{y, z\} \setminus \{q\}$ from T since those jobs are completely covered by $\{x, q\}$. But that means we can remove at least one node from T and still get a valid solution. This contradicts the optimality of T so we can conclude that no triple of jobs in T overlap on a single point. □

Problem 3-3 (*Partially-Sorted) 10 Points

Given an integer k, We say that an array $A[1..n]$ is *partially-sorted* if it can be divided into n/k blocks, each of size k (you may assume that n is always multiples of k), such that the elements in each block are larger than the elements in earlier blocks, and smaller than elements in later blocks. The elements within each block need not be sorted.
For example, the following array is partially sorted for $k = 3$:

1	4	2	7	6	8	10	11	9	15	13	16

(a) Prove that any comparison-based partially-sorting algorithm requires $\Omega(n \log(n/k))$ comparisons in the worst case by using the decision tree method. (5 Points)

Solution: We consider the maximum number of leaves in the decision. There are $n!$ permutations for n numbers, but within each block of size k, the $k!$ possible permutations inside the block are irrelevant. Hence, the maximum number of leaves is:

$$\frac{n!}{[(k)!]^{n/k}} .$$

By using Stirling's approximation, we know that the height of the decision tree is

$$\Omega\Big(\ln \frac{n!}{[(k)!]^{n/k}}\Big) = \Omega(n \log(n/k)) .$$

□

(b) Give an algorithm that completely sorts an already *partially-sorted* array in $O(n \log k)$ time. (5 Points)

Solution: Simply sort each blocks using merge sort (or whichever comparison-based sorting algorithm of your favourite that runs in $O(n \log n)$ time). The running time would simply be $O(k \log k)$ for each block, and in total $O(n \log k)$ for n/k blocks.

□

Problem 3-4 (*Ski and Skier) 10 Points

Consider the following problem. The input consists of n skiers with heights $p_1, \ldots, p_n$, and n skies with heights $s_1, \ldots, s_n$. The problem is to assign each skier a ski to minimize the average difference

between the height of a skier and his/her assigned ski. That is, if the i-th skier is given the $\alpha(i)$-th ski, then you want to minimize:

$$\frac{1}{n}\sum_{i=1}^{n} |p_i - s_{\alpha(i)}| \ .$$

We suggest two greedy algorithms to solve this problem. Only one of these algorithms is correct.

Algorithm A: Find the skier and ski whose absolute height difference is smallest. Assign this skier this ski. Repeat the process until every skier is assigned a ski.

Algorithm B: Give the shortest skier the shortest ski, give the second shortest skier the second shortest ski, give the third shortest skier the third shortest ski, etc.

(a) Which of the Algorithms A or B is *incorrect*? Also, give a counter-example for this algorithm for $n = 2$ formatted as in below. (3 Points)

Counter-example:

$$p_1 = _____, \quad p_2 = _____, \quad s_1 = _____, \quad s_2 = _____$$

$$\text{Greedy average difference } = \frac{1}{2}(____ + ____) = ____$$

$$\text{Optimal average difference } = \frac{1}{2}(____ + ____) = ____$$

Solution: Algorithm A is incorrect. Any example with $p_1 \ll s_1 < p_2 \ll s_2$ works.

Counter-example:

$$p_1 = 1, \quad p_2 = 7, \quad s_1 = 5, \quad s_2 = 11$$

$$\text{Greedy average difference } = \frac{1}{2}(2 + 10) = 6$$

$$\text{Optimal average difference } = \frac{1}{2}(4 + 4) = 4$$

□

(b) Which of the Algorithms A or B is *correct*? Convince yourself that the algorithm is correct for $n = 2$ (you do not need to prove this), and use this and the *local swap* argument to prove the correctness of the algorithm for all n.

Prove that it is correct. (7 Points)

Solution: Algorithm B is correct. We assume the base case that the algorithm is correct for $n = 2$. Now, assume that there is an optimal solution that is not found by the greedy algorithm. Assume that

$$p_1 \le p_2 \le \cdots \le p_n \ .$$

Now any solution to the problem corresponds to finding a permutation $\alpha : \{1, 2, \ldots, n\} \to \{1, 2, \ldots, n\}$ of the skis that minimizes the given sum. For any permutation α, we let $I(\pi)$ be the number of inversions, i.e., the number of pairs (i, j) such that $i < j$ and $s_{\alpha(i)} > s_{\alpha(j)}$.

Assume that the greedy solution does not give the optimal solution. Then, we consider the optimal solution α^* that minimizes the number of inversions. Consider any pair $(i, i+1)$ such that $\alpha^*(i) > \alpha^*(i+1)$. We define a new permutation β such that $\beta(i) = \alpha^*(i+1)$, $\beta(i+1) = \alpha^*(i)$, and $\beta(j) = \alpha^*(j)$, otherwise. Then, we have the following:

- The permutation β has exactly 1 inversion less than α, i.e., $I(\beta) = I(\alpha^*) - 1$.
- By the optimality of the algorithm for $n = 2$,

$$\frac{1}{n}\sum_{i=1}^{n} |p_i - s_{\beta(i)}| \leq \frac{1}{n}\sum_{i=1}^{n} |p_i - s_{\alpha^*(i)}| \,.$$

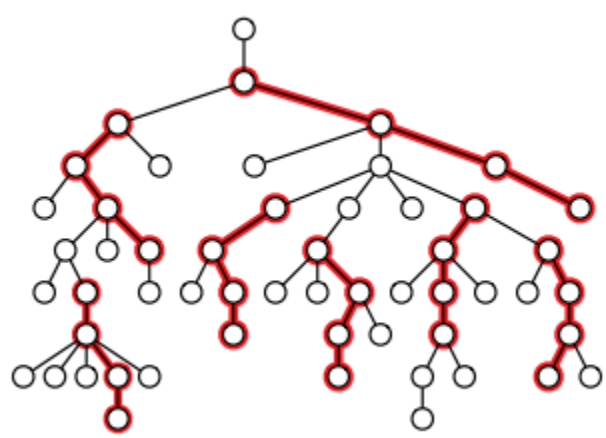
This contradicts the fact that that α^* is an optimal solution that minimizes the number of inversions. □

Problem 3-5 (Paths on Tree)

Given a rooted tree T with n vertices and an integer $0 < k \leq n-1$. A *path* of length k is defined as follows: choose a starting vertex, travel towards the root (in other words, keep visiting the parent vertex) until $k+1$ vertices (including the starting one) have been visited, the $k+1$ visited vertices form a path. Two paths are disjoint if they don't share any vertex.

(a) Describe and analyse a greedy algorithm to compute the maximum number of disjoint paths in T, where each path has length k. Do **not** assume that T is a binary tree. For example, given the tree below as input and $k = 3$, your algorithm should output 8 as the solution. (8 Points)

Figure 1: Seven disjoint paths of length $k = 3$. This is **not** the largest set of disjoint paths.



Solution: Notice that each path is uniquely defined by its starting vertex. The greedy algorithm proceeds as follows: iterate through vertices from highest depth (i.e. furthest away from the root) to lowest depth (i.e. the root), add the path defined by starting at that vertex to the solution if there is no conflict to paths that has already been chosen.

We note that sorting the vertices by depth can be achieved by BFS which takes $O(n)$ time. The procedure above takes $O(k)$ time for a single vertex and in total $O(nk)$ time.

It remains to show correctness. Let V denote the set of vertices. For any vertex $v \in V$, let $P(v)$ denote the path that starts from vertex v and $d(v)$ denote the depth of vertex v. Suppose that $S^* \subset V$ is an optimal solution and let $S \subset V$ be the solution returned by our greedy algorithm. We shall show that $|S| \geq |S^*|$. In particular, we will establish an injective mapping $\pi : S^* \to S$.

Let's go through the vertices $v \in S^*$ from highest depth (i.e. the same order as we run the greedy algorithm). There are two cases:

1. $v \in S$. Then $\pi(v) := v$.
2. $v \notin S$. Consider what happens in the greedy algorithm when it examines v. The reason why $v \notin S$ has to be that the greedy algorithm **already** adds a $u \in S$ such that $P(v)$ intersects with $P(u)$. Hence, $d(u) \geq d(v)$, indicating that $P(v)$ contain the last vertex of $P(u)$. Let $\pi(v) := u$ in this case.

   Clearly $u \notin S^*$ since S^* is a valid solution with disjoint paths. Therefore, for any $v' \in S^*$ that satisfies $\pi(v') = u$, we know $P(v')$ contains the last vertex of $P(u)$. Hence $P(v')$ intersects with $P(v)$ and $v' = v$. The mapping π is indeed injective.

With that, we conclude that $|S^*| \leq |S|$ and the greedy algorithm is correct. □

We also provide another proof that uses the local swap argument.

Solution: As explained in the solution above, the set of paths can be represented by the set of starting vertices. Similar to what we saw in the lecture, let the optimal solution be $O = \{o_1, \ldots, o_m\}$ and the greedy solution be $G = \{g_1, \ldots, g_k\}$. We assume that they are all sorted by depth from high to low (i.e. in the order of how the greedy algorithm examines the vertices). Define the $dist(O) = m - i$ where $i + 1$ is the smallest index such that $o_{i+1} \neq g_{i+1}$. And clearly for any solution set S, $dist(S) = 0$ if and only if $S = G$. Let O^* be the solution set by adding g_{i+1} to O and removing any vertices that conflict with g_{i+1} from O. We claim that O^* has size m, in other words, g_{i+1} conflict with exactly 1 path from $o_{i+2}, \ldots, o_m$. It is not possible to have no conflict since it violates our assumption that O is optimal. Any path in O that conflicts with g_{i+1} has depth smaller or equal to the depth of g_{i+1}. Hence, they must share the last vertex of the path defined by g_{i+1}, indicating that there is at most one such path in O which is a valid solution set.

The local swap argument does follow as we keep swapping until we finally reach G, indicating that $k = m$. □

(b) Now suppose each vertex in T has an associated reward, and your goal is to maximize the total reward of the vertices in your paths, instead of the total number of paths. Show that your greedy algorithm does **not** always return the optimal reward. [Hint: Give a counter example.] (1 Point)

Solution: Consider a tree with $n = 3$ vertices that forms a line and the root has the highest reward. If $k = 1$, the path returned by the greedy algorithm would not include the root, which is suboptimal. □

(c) Describe an efficient algorithm to compute the maximum possible reward as described in part (b). Prove that your algorithm is correct. (8 Points)

Solution: We use $C(v)$ to denote the set of children's of vertex v and $R(v)$ to dentoe the associated reward at v. For $v \in V, 0 \leq i \leq k$, let $f[v,i]$ denote the maximum reward achievable at the subtree rooted at v, and v being the $i+1$-th vertex of a chosen path. For simplicity, we force $f[v,k]$ to include the case where v is not part of any chosen path. For v to be the $i+1$-th vertex of a chosen path, it has to choose a child which is the i-th vertex of a chosen path, and collect whatever rewards from other children. In particular, we have the recursive formula as

$$
\begin{aligned}
f[v,0] &:= R(v) + \sum_{c \in C(v)} f[c,k] \\
f[v,i] &:= R(v) + \max_{c \in C(v)} \Big(f[c,i-1] + \sum_{c' \in C(v), c' \neq c} f[c,k] \Big) && 0 < i \leq k \\
f[v,k] &:= \max \Big(f[v,k], \sum_{c \in C(v)} f[c,k] \Big) . && \text{the extra case}
\end{aligned}
$$

We can then iterate through the vertex according to the depth since $f[v,i]$ depends only on the children of v.

As for the running time, for any fixed i, $f[v,i]$ is only referenced for a constant number of times (when we computed it, and when it is referenced by its parent). The total running time is thus $O(nk)$. (There is some flavour of amortised analysis here.) $\square$